

Week 1: Introduction to Reinforcement Learning

Lecture Notes & Core Concepts Write-Up

1. What is Reinforcement Learning?

Reinforcement Learning (RL) is a computational framework for learning through interaction. Unlike supervised learning, which requires labeled examples, or unsupervised learning, which discovers structure in data, RL is grounded in a fundamentally different paradigm: an agent learns by taking actions in an environment and receiving feedback in the form of rewards or penalties. The goal is not to mimic a teacher but to discover, through trial and error, a strategy that maximizes cumulative reward over time.

Formally, RL is best understood as a sequential decision-making problem. At each timestep, the agent observes the current state of the environment, selects an action, transitions to a new state, and receives a scalar reward signal. This cycle repeats, and the agent must learn from experience alone — no ground truth labels, no model of how the world works. This makes RL both powerful and challenging: it can solve problems where the rules are unknown, but it must do so with often sparse and delayed feedback.

2. Agent–Environment Interaction

The agent–environment loop is the backbone of RL. At every discrete timestep t , the interaction proceeds as follows:

- The agent observes state S_t from the environment.
- It selects an action A_t according to its current policy.
- The environment transitions to a new state S_{t+1} and emits a reward R_{t+1} .
- The agent updates its knowledge using this experience tuple $(S_t, A_t, R_{t+1}, S_{t+1})$.

The environment can be a physical system (a robot arm), a simulator (a video game), or an abstract process (a financial market). The agent's only window into the world is the state signal and the reward it receives. Importantly, the agent does not know the rules of the environment in advance; it must infer what is beneficial from the reward signal alone.

3. Return

The return G_t is the total accumulated reward the agent collects from timestep t onward. For an undiscounted, finite-horizon problem:

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

In practice a **discount factor** $\gamma \in [0, 1)$ is introduced, down-weighting rewards that lie further in the future:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Discounting ensures the sum converges for infinite-horizon problems (when $\gamma < 1$) and encodes the idea that immediate rewards are more certain than distant ones. A γ near 1 makes the agent far-sighted; a γ near 0 makes it myopic.

4. Policy

A **policy** π is the agent's decision-making rule — a mapping from states to actions (or distributions over actions):

- **Deterministic:** $\pi(s) = a$ — one specific action per state.
- **Stochastic:** $\pi(a|s) = P[A_t = a | S_t = s]$ — a probability distribution over actions.

$$\pi(a | s) = P[A_t = a \mid S_t = s]$$

The central objective of RL is to find an **optimal policy** π^* that maximises expected return from every state. Stochastic policies are especially useful during training because they naturally encode exploration.

5. Value Functions

Value functions quantify how good a state (or a state– action pair) is under a given policy. They are central to most RL algorithms.

5.1 State-Value Function $V_{\pi}(s)$

The expected return obtained by starting in state s and following policy π thereafter:

$$V_{\pi}(s) = \mathbb{E}_{\pi} [G_t \mid S_t = s]$$

5.2 Action-Value Function $Q_{\pi}(s, a)$

The expected return when starting in state s , taking action a , and then following π :

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi} [G_t \mid S_t = s, A_t = a]$$

The two functions are related by:

$$V_{\pi}(s) = \sum_a \pi(a | s) Q_{\pi}(s, a)$$

Knowing the optimal Q-function $Q^*(s, a)$ immediately gives the optimal policy: $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$. This relationship underpins Q-learning and Deep Q-Networks (DQN).

6. Exploration vs. Exploitation

A core tension in RL is the **exploration– exploitation dilemma**:

- **Exploit**: choose the currently best-known action — maximises short-term reward but may miss better undiscovered strategies.
- **Explore**: try a less familiar action — may sacrifice immediate reward but can reveal superior long-run strategies.

Consider a multi-armed bandit: always pulling the best-known lever means never discovering whether another lever pays out more. But endless exploration wastes reward. Neither extreme is optimal — a balance must be struck.

The canonical solution is the **ϵ -greedy strategy**: with probability ϵ choose a random action (explore); with probability $1 - \epsilon$ choose the greedy best action (exploit). More principled methods include Upper Confidence Bound (UCB) — which directs exploration toward high-uncertainty actions — and Thompson Sampling. In deep RL, entropy regularisation can encourage sustained exploration throughout training.

7. Formalizing RL as Optimization

RL problems are formally cast as **Markov Decision Processes (MDPs)**, defined by the tuple:

$$(S, A, P, R, \gamma)$$

- **S** — set of all states
- **A** — set of all actions
- **$P(s' | s, a)$** — transition probability (environment dynamics)
- **$R(s, a)$** — expected reward after taking action a in state s
- **γ** — discount factor

The Markov property — the future depends only on the current state, not the full history — is the key structural assumption enabling tractable solution methods.

The optimization objective is to find a policy π^* that maximises the expected discounted return from the starting state:

$$\pi^* = \operatorname{argmax}_{\pi} \mathbb{E}_{\pi} [G_0]$$

This can be solved via dynamic programming (when P and R are known), model-free methods

such as Q-learning or policy gradients (when they are not), or model-based approaches that first learn the environment dynamics and then plan within them.

Sources: Sutton & Barto (2020), Ch. 1 • David Silver RL Lecture 1 • OpenAI Spinning Up